06

Stripe in production

Webhook reliability, refunds, subscription edge cases, and the test-to-live failures nobody warns you about.

SUBJECT	PAYMENTS · WEBHOOKS	READER	SOLO BUILDERS ON AI TOOLS
AUTHOR	MICAH JONES	STATUS	CHAPTER SIX OF TEN
TIME	A TEN-MINUTE READ	REV	2026.08

§ 06.1

The stakes change here

Every failure in this book so far cost you embarrassment or a weekend. This chapter's failures cost money and trust, in public, one customer at a time. A broken upload annoys someone. A customer who paid and got nothing tells everyone.

The good news is the same shape as chapter five's: you are not inventing payment security. Stripe carries the hard parts, and your job reduces to three sentences.

> Your app never touches money. It sends people to Stripe, believes only what Stripe's signed webhooks say happened, and keeps its own "paid" column as a mirror of Stripe's records. Every payment bug in solo builds is a violation of one of those three sentences.

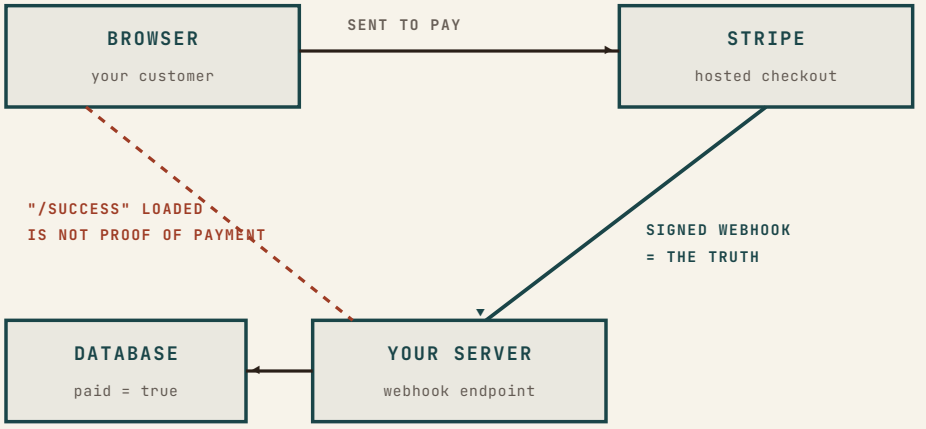
§ 06.2

The flow that survives

Use Stripe's hosted checkout page, not a card form of your own. The AI will happily build you a beautiful custom card form, and it will be a beautiful liability: card data routed through your code drags you into compliance territory an entire team would respect, and conversion is worse than the page buyers already recognize. Send them to Stripe; Stripe sends them back.

SOURCE OF TRUTH

> For money state, Stripe's records are the fact and your database is a cache of the fact. When the two disagree, Stripe is right, and your job is to resync, not to argue.



MONEY STATE FLOWS ONE WAY: STRIPE, THEN SERVER, THEN DB

The diagram has one villain, and it is the dashed line: the success page. When Stripe redirects your customer to `/success`, that page load proves navigation, not payment. Tabs close. Networks drop. And anyone can type `/success` into a URL bar. The single most common money bug in AI-built apps is granting access on that page load, because it demos perfectly and fails in production in both directions: paying customers who closed the tab get nothing, and curious visitors who typed the URL get everything.

Access is granted in exactly one place: the webhook handler.

§ 06.3

Webhooks, the load-bearing wall

A webhook is Stripe calling your server: an HTTP POST to an endpoint you registered, carrying “checkout completed” or “charge refunded.” Chapter three drew this arrow with the label “verified webhooks,” and now it earns its keep. Four rules make the wall hold.

Verify the signature, every time. Your endpoint is a public URL. Without the signature check, anyone on earth can POST “payment succeeded” to it and grant themselves your product. Stripe signs every event with a webhook secret; the check is three lines with the SDK, and the secret is an environment variable, chapter four style.

Welcome retries; process events once. Stripe retries delivery until you answer with a 200, which means the same event can arrive twice. Record processed event IDs and skip

repeats, or your database will double-grant, double-log, and double-email.

Trust the event, not the order. Events can arrive out of sequence. Handle each one by asking Stripe's records what the current state is, rather than assuming the last event you saw told the whole story.

Answer fast, work after. Acknowledge the event, then do the slow parts. An endpoint that dawdles gets retried, and now you are back to rule two. "After" needs no clever machinery at solo scale: save the event to a table, answer, and let a job that runs every minute send the emails and do the slow writes.

§ 06.4

Test-to-live: the five silent swaps

Stripe's test mode is a parallel universe: same dashboard, same API shape, fake money. The day you flip to live, five things silently do not come along, and each one fails without an error message.

1. **The keys.** `sk_test_` and `sk_live_` both work perfectly, each in its own universe. Live site with test keys means imaginary revenue; a test script with live keys means real charges.
2. **The webhook endpoint.** Registered separately per mode. Your test webhook does not fire for live events; live needs its own registration.
3. **The webhook secret.** The new live endpoint gets a new signing secret, and the old one keeps verifying test events only. Signature failures after go-live are this, almost every time.
4. **The prices.** Products and price IDs are per mode. The `price_...` your checkout references in test does not exist in live until you recreate it.
5. **The redeploy.** All four above live in environment variables and configuration, and chapter four already taught the rule: installed is not live. Change, deploy, then test.

FIELD NOTE

Local development wrinkle: Stripe cannot call localhost. The Stripe CLI forwards live test events to your machine while you build. It also hands you a different signing secret than production uses, which is the next section's trap in miniature.

§ 06.5

Refunds, disputes, subscriptions

Refunds at solo scale are a dashboard click, and that is fine. The part that belongs in your code is the echo: the click fires

a `charge.refunded` webhook, and your handler revokes what the payment granted. A refund policy your code doesn't hear about is a discount, not a policy.

Disputes are the one email you never ignore. When a cardholder disputes a charge, silence is an automatic loss. Submit the evidence Stripe asks for: what was bought, when it was delivered, the logs that show usage. Solo builders lose most disputes they contest and all of the ones they don't; the difference funds the habit.

Subscriptions add exactly three edge cases worth engineering for on day one. Failed renewals: cards expire, and Stripe retries on a schedule, so treat `past_due` as a grace period with a friendly email, not an instant cutoff. Cancellations: "cancel at period end" is almost always what your customer means; immediate cancellation with no refund is how you earn the dispute above. And status: your app reads subscription state from one place, the column your webhooks maintain, never from assumptions about what the customer probably did.

FROM THE BUILD LOG ENTRY · 2026-08

The buy button I refused to ship

The sales page for this manual went live weeks before its checkout existed. The temptation was strong to ship the $149 button anyway, wired to nothing or to something untested, because a page without a buy button feels unfinished.

It shipped as a waitlist instead: leave an email, get chapter one free, be told the day it opens. The reasoning is this chapter in one sentence: money UI ships last, after the money path is verified end to end, because the exact moment someone decides to pay you is the worst possible moment to show them a lie.

The first real charge on any build should be your own card, at the live URL, watched all the way through: checkout, webhook, database flip, confirmation email. Then refund yourself and watch the refund echo through the same pipe. That is the two-account test of money, and it costs you nothing but the fee.

Pre-flight: money

PRE-FLIGHT · STRIPE — **RUN TONIGHT**

- [ ] **Hosted checkout only.** Your server never sees a card number, and the AI's beautiful custom card form stays in the parking lot.
- [ ] **Access is granted by the verified webhook, never by the success page.** Signature checked, event IDs deduped, retries welcomed.
- [ ] **Make the five live-mode swaps deliberately:** live keys, live endpoint, its new secret, live price IDs, then redeploy. Each is an env change, and installed is not live.
- [ ] **Wire the refund echo.** `charge.refunded` revokes what payment granted. Test it with a real refund to yourself.
- [ ] **Pay yourself once, live, and watch the whole pipe:** checkout, webhook, database, email. Then refund it. The fee is the cheapest audit you will ever buy.

Money in, money safe, money honest. What's left is the paperwork the outside world attaches to it, and when that paperwork actually applies to you: HIPAA, SOC 2, GDPR, and the acronyms in between.

NEXT · CHAPTER 07

Compliance, when it matters

HIPAA, SOC 2, GDPR. When you genuinely need them, when you don’t, and what compliant actually requires.